Monitor API

class **mini.apis.api_observe.ObserveSpeechRecognise** [source]

Bases: `mini.apis.base_api.BaseEventApi`

Monitor speech recognition api

Monitor voice recognition events, and the robot reports the text after voice recognition

#SpeechRecogniseResponse.text: The recognized text

#SpeechRecogniseResponse.isSuccess: Is it successful

#SpeechRecogniseResponse.resultCode: Return code

async **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

stop() [source]

Stop speech recognition, stop monitoring speech recognition

Returns: None

class **mini.apis.api_observe.ObserveFaceDetect** [source]

Bases: `mini.apis.base_api.BaseEventApi`

Monitor the number of faces api

Monitor the number of faces events, and the robot reports the number of detected faces

Single detection timeout time 1s, detection interval 1s

#FaceDetectTaskResponse.count(int): the number of faces

#FaceDetectTaskResponse.isSuccess: Is it successful, True or False

#FaceDetectTaskResponse.resultCode: Return code

async **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

stop() [source]

Stop detecting the number of faces, stop monitoring the number of faces

Returns: None

---

class **mini.apis.api_observe.ObserveFaceRecognise** [source]

Bases: `mini.apis.base_api.BaseEventApi`

Monitor face recognition api

Monitor face recognition events, and the robot reports the recognized face information (array)

If it is a registered face, return face details: id, name, gender, age

If it is a stranger, return name: “stranger”

Single detection timeout time 1s, detection interval 1s

FaceRecogniseTaskResponse.faceInfos: [FaceInfoResponse] face information array

FaceInfoResponse.id, FaceInfoResponse.name,FaceInfoResponse.gender,FaceInfoResponse.age: face details

FaceRecogniseTaskResponse.isSuccess: Is it successful, True or False

FaceRecogniseTaskResponse.resultCode: Return code

async **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

stop() [source]

Stop face recognition, stop monitoring face recognition

Returns: None

---

class **mini.apis.api_observe.ObserveInfraredDistance** [source]

Bases: `mini.apis.base_api.BaseEventApi`

Monitor infrared distance api

Monitor infrared distance events, the robot reports the detected infrared distance to the nearest obstacle in front of you

Detection cycle 1s

ObserveInfraredDistanceResponse.distance: infrared distance

async **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

stop() [source]

Stop infrared distance detection, stop monitoring infrared distance

Returns: None

---

class **mini.apis.api_observe.RobotPosture**(*value*) [source]

Bases: `enum.Enum`

Robot pose

STAND: standing

SPLITS_LEFT: Left lunge

SPLITS_RIGHT: Right lunge

SIT_DOWN: sit down

SQUAT_DOWN: Squat down

KNEELING: kneel down

LYING: lying on the side

LYING_DOWN: lie flat

SPLITS_LEFT_1: Left split

SPLITS_RIGHT_2: Right split

BEND: Bent over

STAND= *1*

SPLITS_LEFT= *2*

SPLITS_RIGHT= *3*

SIT_DOWN= *4*

SQUAT_DOWN= *5*

KNEELING= *6*

LYING= *7*

LYING_DOWN= *8*

SPLITS_LEFT_1= *9*

SPLITS_RIGHT_2= *10*

BEND= *11*

---

class **mini.apis.api_observe.ObserveRobotPosture** [source]

Bases: `mini.apis.base_api.BaseEventApi`

Monitor robot attitude api

Monitor robot posture change events, and the machine reports the current posture RobotPosture (when the posture changes)

#ObserveFallClimbResponse.status: Machine posture, the value corresponds to RobotPosture

async **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

stop() [source]

Stop robot pose detection, stop monitoring robot pose

Returns: None

---

class **mini.apis.api_observe.HeadRacketType**(*value*) [source]

Bases: `enum.Enum`

Head shot

SINGLE_CLICK: click

LONG_PRESS: Long press

DOUBLE_CLICK: double click

> **SINGLE_CLICK**= *1*

> **LONG_PRESS**= *2*

> **DOUBLE_CLICK**= *3*

---

class **mini.apis.api_observe.ObserveHeadRacket** [source]

Bases: `mini.apis.base_api.BaseEventApi`

Listen to the head event api

Monitor the head event, and report the head type when the robot's head is tapped

ObserveHeadRacketResponse.type: Racket head type, HeadRacketType

> *async* **execute**() [source]

Send instructions

After serializing the supported message, write it to the socket Implemented by subclasses

> **stop**() [source]

Stop the detection of the robot head event, stop listening to the robot head event

Returns: None